Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

Actividad: Biblioteca.

Enlace al repositorio de la página: GitHub - rodrigo-cyber17

Actividad: Gestión de biblioteca con Hibernate.

En esta actividad van a desarrollar una aplicación Java que gestiona una base de datos de una biblioteca utilizando Hibernate como framework ORM. Implementarán el mapeo objeto-relacional mediante anotaciones JPA y realizarán operaciones CRUD completas sobre las entidades, prestando especial atención a las relaciones entre tablas y las operaciones en cascada.

Modelo de datos:
El sistema de gestión de biblioteca debe permitir administrar autores, libros y ejemplares físicos. El modelo de datos está compuesto por tres **entidades** principales:
- Autor: Representa a los escritores de los libros.
- Libro: Representa las obras literarias.
- Ejemplar: Representa las copias físicas de cada libro.

Relaciones:
- Un autor puede escribir múltiples libros (1:N)
- Un libro pertenece a un único autor (N:1)
- Un libro puede tener múltiples ejemplares físicos (1:N)
- Un ejemplar pertenece a un único libro (N:1)

Operaciones en cascada:
- Al eliminar un autor, se deben eliminar todos sus libros
- Al eliminar un libro, se deben eliminar todos sus ejemplares
- Al guardar un autor con libros nuevos, los libros deben guardarse automáticamente

ÍNDICE

1. Apartado 1. - Preparación del entorno..... 3

1.1. Apartado 1.1. - Creación de la base de datos.....3

1.2. Apartado 1.2. - Configuración del proyecto Maven.....4

1.3. Apartado 1.3. - Configuración de Hibernate.....7

1.4. Apartado 1.4. - Clase HibernateUtil (paquete com.dam.util).....8

2. Apartado 2. - Implementación de entidades..... 9

3. Apartado 3. - Operaciones Create.....12

4. Apartado 4. - Operaciones Read..... 14

5. Apartado 5. - Operaciones Update.....18

6. Apartado 6. - Operaciones Delete..... 21

7. Apartado 7. - Clase principal y menú..... 24

8. Apartado 8. - Explicación del programa.....25

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

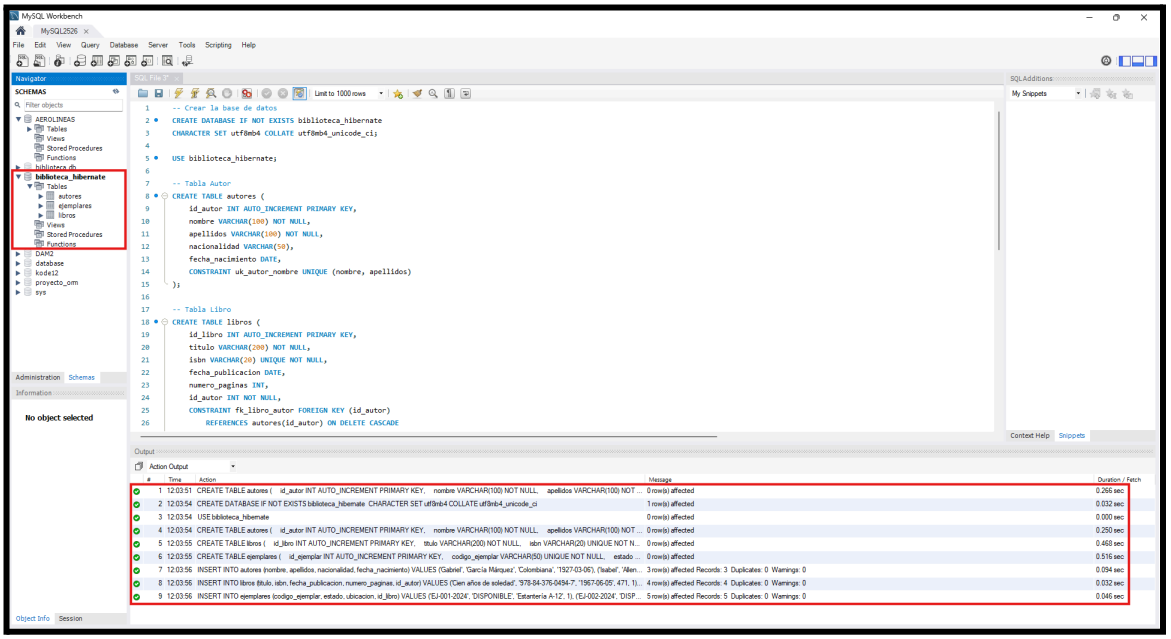
---

1. Paso

"Preparación del entorno".

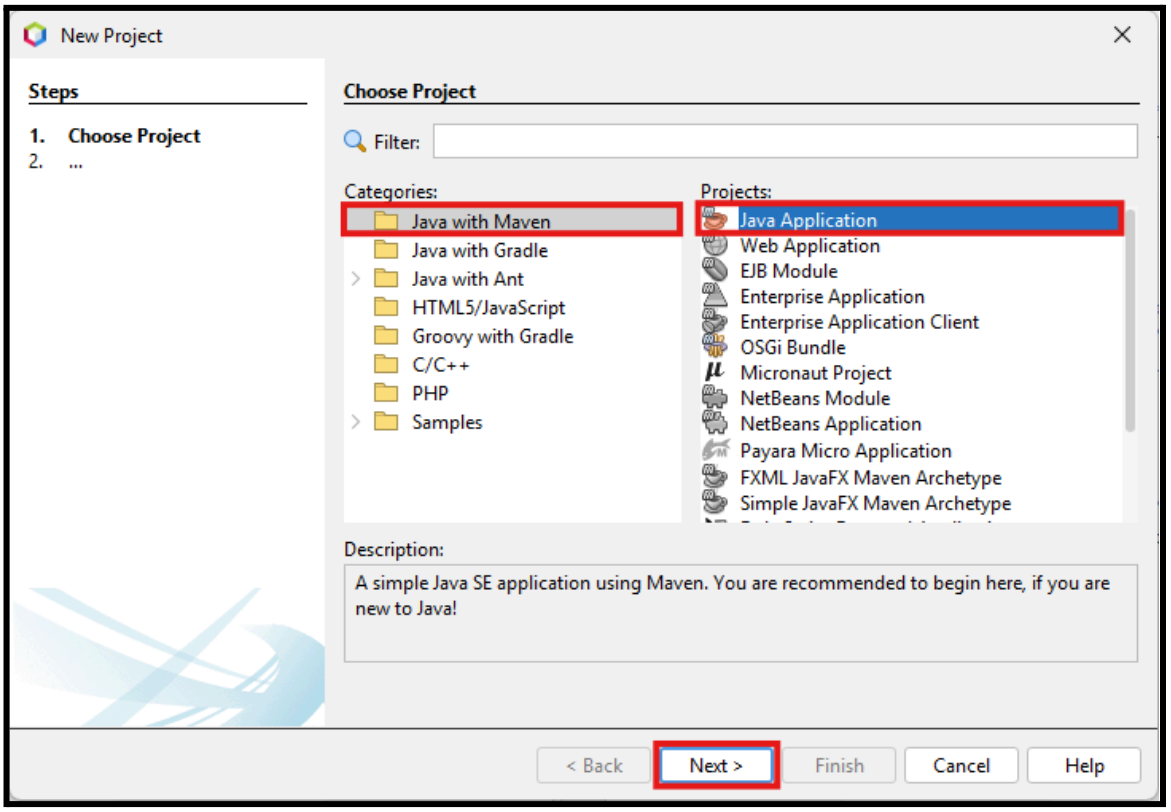
1.1. Paso

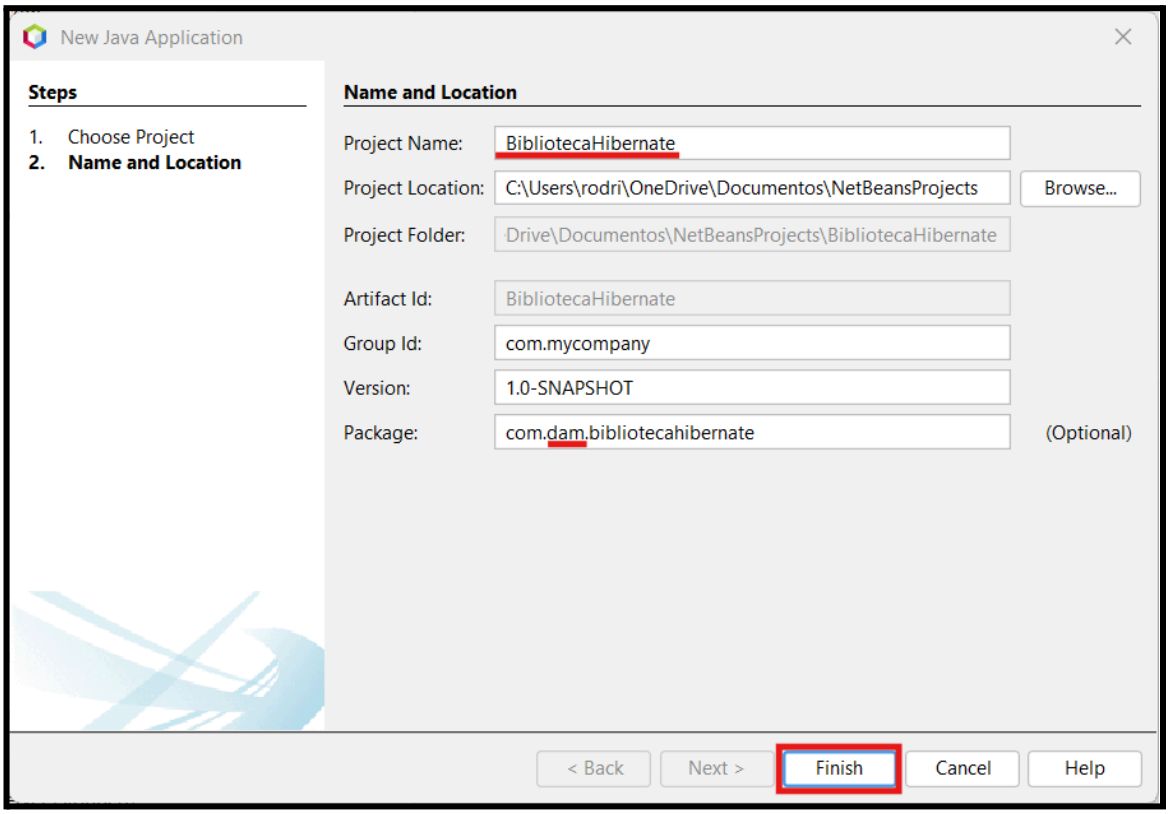
"Creación de la base de datos".



1.2. Paso

"Configuración del proyecto Maven".




New Java Application
Steps
1. Choose Project
2. Name and Location
Name and Location
Project Name: BibliotecaHibernate
Project Location: C:\Users\rodri\OneDrive\Documentos\NetBeansProjects
Browse...
Project Folder: Drive\Documentos\NetBeansProjects\BibliotecaHibernate
Artifact Id: BibliotecaHibernate
Group Id: com.mycompany
Version: 1.0-SNAPSHOT
Package: com.dam.bibliotecahibernate
(Optional)
< Back
Next >
Finish
Cancel
Help

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

pom.xml [GestionBiblioteca]

Source Graph Effective History

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchem
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.mycompany</groupId>
    <artifactId>GestionBiblioteca</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>23</maven.compiler.release>
        <exec.mainClass>com.dam.bibliotecahibernate.GestionBiblioteca</exec.mainClass>
    </properties>
    <dependencies>
        <!-- MySQL Connector -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>9.3.0</version>
        </dependency>

        <!-- Hibernate -->
        <dependency>
            <groupId>org.hibernate</groupId>
            <artifactId>hibernate-core</artifactId>
            <version>5.6.15.Final</version>
        </dependency>

        <!-- JPA -->
        <dependency>
            <groupId>javax.persistence</groupId>
            <artifactId>javax.persistence-api</artifactId>
            <version>2.2</version>
        </dependency>
    </dependencies>
    <name>BibliotecaHibernate</name>
</project>
```

Output - Build (BibliotecaHibernate)

```
--- install:3.1.2:install (default-install) @ GestionBiblioteca ---
Installing C:\Users\rodri\OneDrive\Documentos\NetBeansProjects\Biblioteca
Installing C:\Users\rodri\OneDrive\Documentos\NetBeansProjects\Biblioteca
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  3.443 s
Finished at: 2025-11-09T22:34:39Z
------------------------------------------------------------------------
```

1.3. Paso

"Configuración de Hibernate".

hibernate.cfg.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">


<hibernate-configuration>
    <session-factory>
        <!-- Configuración de la base de datos -->
        <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/biblioteca_hibernate</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">root</property>

        <!-- Dialecto de la base de datos (Dialecto SQL) -->
        <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>

        <!-- Pool de conexiones -->
        <property name="hibernate.connection.pool_size">10</property>

        <!-- Mostrar SQL en consola -->
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.format_sql">true</property>

        <!-- Gestión automática del esquema -->
        <property name="hibernate.hbm2ddl.auto">validate</property>

        <!-- Contexto de sesión actual -->
        <property name="hibernate.current_session_context_class">thread</property>

        <!-- Mapeo de clases de entidad -->
            <!-- Modificación para JPA -->
        <!--
        <mapping resource="Autor.xml"/>
        <mapping resource="Ejemplar.xml"/>
        <mapping resource="Libro.xml"/>
        -->
        <!-- Mapeo de archivos -->
        <mapping class="com.dam.modelo.Autor"/>
        <mapping class="com.dam.modelo.Ejemplar"/>
        <mapping class="com.dam.modelo.Libro"/>
    </session-factory>
</hibernate-configuration>
```

Output - Build (BibliotecaHibernate)

```
--- install:3.1.2:install (default-install) @ GestionBiblioteca ---
Installing C:\Users\rodri\OneDrive\Documentos\NetBeansProjects\BibliotecaHibernate\pom.xml to C:\Users\rodri\.m2\reposito
Installing C:\Users\rodri\OneDrive\Documentos\NetBeansProjects\BibliotecaHibernate\target\GestionBiblioteca-1.0-SNAPSHOT.
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  3.392 s
Finished at: 2025-11-09T22:53:49Z
------------------------------------------------------------------------
```

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

1.4. Paso

"Clase HibernateUtil (paquete com.dam.util)".

HibernateUtil.java

Source History

```java
/**
 *
 * @author Rodrigo
 */
package com.dam.util;


// Importa de la biblioteca/librería el paquete "JOptionPane".
import javax.swing.JOptionPane;
import org.hibernate.HibernateException;
// Importa de la biblioteca/librería el paquete "SessionFactory".
import org.hibernate.SessionFactory;
// Importa de la biblioteca/librería el paquete "Configuration".
import org.hibernate.cfg.Configuration;

public class HibernateUtil {
    private static final SessionFactory sessionFactory;

    static{
        try{
            sessionFactory = new Configuration().configure().buildSessionFactory();
            /*
            Configuration configuration = new Configuration();
            configuration = configuration.configure();

            StandardServiceRegistryBuilder builder = new StandardServiceRegistryBuilder();
            builder = builder.applySettings(configuration.getProperties());

            SessionFactory sessionFactory = configuration.buildSessionFactory(builder.build());
            */
        } catch(HibernateException tw){
            JOptionPane.showMessageDialog(null, "Error de sesiones", "'Session Factory' creation fai
            throw new ExceptionInInitializerError(tw);
        } catch(Throwable tw){
            JOptionPane.showMessageDialog(null, "Error de sesiones", "'Session Factory' creation fai
            throw new ExceptionInInitializerError(tw);
        }
    }

  // Crea el método "get".
  public static SessionFactory getSessionFactory(){ return sessionFactory; }

  // Crea el método "shutdown".
  public static void shutdown(){ getSessionFactory().close(); }
}
```

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

---

2. Paso

"Implementación de entidades".

Autor.java

```java
/**
 *
 * @author Rodrigo
 */
package com.dam.modelo;


// Importa de todos los paquetes de la biblioteca/librería "persistence".
import javax.persistence.*;
// Importa de todos los paquetes de la biblioteca/librería "LocalDate".
import java.time.LocalDate;
// Importa de todos los paquetes de la biblioteca/librería "ArrayList".
import java.util.ArrayList;
// Importa de todos los paquetes de la biblioteca/librería "List".
import java.util.List;

// Definimos una entidad que representará a un autor de libros en la biblioteca.
    /*
     * 1. Primero mapea a la tabla "autores" en la base de datos.
     * 2. Despues define una relación donde/en la que un autor puede escribir múltiples libros, es decir,
     */
@Entity
@Table(name = "autores")

// Crea la clase "Autor".
public class Autor{
    // ==================== ATRIBUTOS ====================
        // Declara las variables, atributos.
            // (Sustituye al archivo "Autor.hbm.xml").
                /*
                 * 1. Se genera el identificador, "ID", único (clave primaria) de "autor".
                 * 2. Se genera automáticamente mediante estrategia "IDENTITY".
                 * 3. Se 'ancla'/enlaza a la columna de nombre "id_autor".
                 */
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_autor")
    private long id;

                /*
                 * 1. Atributo para el nombre del autor.
                 * 2. Se 'ancla'/enlaza a la columna de nombre "nombre", añadiéndole atributos de campo ob
                 */
    @Column(name = "nombre", nullable = false, length = 100)
    private String nombre;

                /*
                 * Atributo para los apellidos del autor.
```

Entidad 'Autor' (Autor.java).

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

Ejemplar.java

Source History

```java
/**
 *
 * @author Rodrigo
 */
package com.dam.modelo;


// Importa de todos los paquetes de la biblioteca/librería "persistence".
import javax.persistence.*;

// Definimos una entidad que representará a un autor de libros en la biblioteca.
    /*
     * 1. Primero mapea a la tabla "ejemplares" en la base de datos.
     * 2. Despues define una relación donde/en la que un ejemplar pertenece a un único libro, es decir, re
     */
@Entity
@Table(name = "ejemplares")

// Crea la clase "Libro".
public class Ejemplar{
    // ==================== ATRIBUTOS ====================
        // Declara las variables, atributos.
            // (Sustituye al archivo "Ejemplar.hbm.xml").
                /*
                 * 1. Se genera el identificador, "ID", único (clave primaria) de "ejemplar".
                 * 2. Se genera automáticamente mediante estrategia "IDENTITY".
                 * 3. Se 'ancla'/enlaza a la columna de nombre "id_ejemplar".
                 */
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_ejemplar")
    private long id;

                /*
                 * 1. Atributo para el código del ejemplar.
                 * 2. Se 'ancla'/enlaza a la columna de nombre "codigoEjemplar", añadiéndole atributos de
                 */
    @Column(name = "codigo_ejemplar", unique = true, nullable = false, length = 50)
    private String codigoEjemplar;

                /*
                 * Atributo para el estado de disponibilidad del ejemplar.
                 * Se mapea como un "enum" (tipo enumerado) usando "STRING" y se almacena el nombre del "e
                 * Se 'ancla'/enlaza a la columna de nombre "estado", añadiéndole el atributo de máximo 20
                 * Al no anclar a la columna "estado" el atributo "nullable" el campo queda opcional, en
                 *
                 * Los posibles valores del "enum" a utilizar son:
                 *   El valor " DISPONIBLE " Indica que el ejemplar está disponible.
```

1:1 INS Windows (CRLF)

Entidad 'Ejemplar' (Ejemplar.java).

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

Libro.java

Source History

```java
/**
 *
 * @author Rodrigo
 */
package com.dam.modelo;


// Importa de todos los paquetes de la biblioteca/librería "persistence".
import javax.persistence.*;
// Importa de todos los paquetes de la biblioteca/librería "LocalDate".
import java.time.LocalDate;
// Importa de todos los paquetes de la biblioteca/librería "ArrayList".
import java.util.ArrayList;
// Importa de todos los paquetes de la biblioteca/librería "List".
import java.util.List;

// Definimos una entidad que representará a un libro correspondiente de ejemplares en la biblioteca.
    /*
     * 1. Primero mapea a la tabla "libros" en la base de datos.
     * 2. Despues define una relación donde/en la que un libro puede corresponder a múltiples ejemplares,
     */
@Entity
@Table(name = "libros")

// Crea la clase "Libro".
public class Libro{
    // ==================== ATRIBUTOS ====================
        // Declara las variables, atributos.
            // (Sustituye al archivo "Libro.hbm.xml").
                /*
                 * 1. Se genera el identificador, "ID", único (clave primaria) de "libro".
                 * 2. Se genera automáticamente mediante estrategia "IDENTITY".
                 * 3. Se 'ancla'/enlaza a la columna de nombre "id_libro".
                 */
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_libro")
    private long id;

                /*
                 * 1. Atributo para el título del libro.
                 * 2. Se 'ancla'/enlaza a la columna de nombre "titulo", añadiéndole atributos de campo ol
                 */
    @Column(name = "titulo", nullable = false, length = 200)
    private String titulo;

                /*
                 * Atributo para el código ISBN del libro.
```

1:1 INS Windows (CRLF)

Entidad 'Libro' (Libro.java).

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

---

3. Paso

"Operaciones Create".

GestionBiblioteca.java

```java
// Crea el método "crearAutorConLibros". Su función será ir añadiendo objetos, autores, (asociándole también dos nuevos l
    /*
     * Crea un nuevo autor con dos libros asociados, y cada libro con sus ejemplares. Gracias a las operaciones en cascad
     */
public void crearAutorConLibros(){
    // Se abre una sesión de Hibernate para interactuar con la base de datos.
    session = HibernateUtil.getSessionFactory().openSession();
    //transaction = null;

    try{
        // Se abre/inicia una nueva transacción.
        transaction = session.beginTransaction();

        System.out.println("\n\n\tCreando el autor junto a sus libros y respectivos ejemplares...");

        // Bloque de código para crear el autor.
        Autor autor = new Autor(
                nombre: "Javier",
                apellidos: "Marías",
                nacionalidad: "Española",
                fechaNacimiento: LocalDate.of(year: 1951, month:9, dayOfMonth:20)
        );

        // Bloque de código para crear el libro 1.
        Libro libro1 = new Libro(
                titulo: "Corazón tan blanco",
                isbn: "978-84-204-2954-3",
                fechaPublicacion: LocalDate.of(year: 1992, month:3, dayOfMonth:1),
                numeroPaginas: 368
        );

        // Bloque de código para crear ejemplares.
        Ejemplar ejemplar1 = new Ejemplar(codigoEjemplar: "EJ-009-2024", estado: EstadoEjemplar.DISPONIBLE, ubicacion: "Estantería E-
        Ejemplar ejemplar2 = new Ejemplar(codigoEjemplar: "EJ-010-2024", estado: EstadoEjemplar.DISPONIBLE, ubicacion: "Estantería E-

        // Bloque de código para añadir los ejemplares creados al libro 1.
        libro1.addEjemplar(ejemplar: ejemplar1);
        libro1.addEjemplar(ejemplar: ejemplar2);

        // Bloque de código para crear el libro 2.
        Libro libro2 = new Libro(
                titulo: "Mañana en la batalla piensa en mí",
                isbn: "978-84-204-2955-0",
                fechaPublicacion: LocalDate.of(year: 1994, month:5, dayOfMonth:5),
                numeroPaginas: 464
        );
```

36:6 INS Unix (LF)

Operación 'crear autor/es' (Método "crearAutorConLibros()") (GestionBiblioteca.java).

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

GestionBiblioteca.java

Source History

```java
    // Crea el método "agregarEjemplarALibroExistente". Su función será ir añadiendo objetos, ejemplares, correspondientes a
        /*
         * Busca un libro por su código ISBN y le añade un nuevo ejemplar. Este nuevo ejemplar se crea con estado "DISPONIBLE
         */
    public void agregarEjemplarALibroExistente(String isbn, String codigoEjemplar){
        // Se abre una sesión de Hibernate para interactuar con la base de datos.
        session = HibernateUtil.getSessionFactory().openSession();
        //transaction = null;

        try{
            // Se abre/inicia una nueva transacción.
            transaction = session.beginTransaction();

            System.out.println("\n\n\tBuscando un ejemplar/es por su código ISBN {" + isbn + "}...");

            // Bloque de código para buscar un libro por su ISBN (código) usando "HQL" (Hibernate Query Language).
                // ":isbn" Es el valor real que espera del parametro que pasaremos "setParameter" (" "isbn" " →(=) isbn).
                    // Forma 1.
            //Libro libro = session.createQuery("FROM Libro l WHERE l.isbn = :isbn", Libro.class).setParameter("isbn", isbn).
                    // Forma 2.
            String hql = "FROM Libro l WHERE l.isbn = :isbn";
            Query<Libro> query = session.createQuery(string: hql, type: Libro.class);
            query.setParameter(string: "isbn", o: isbn);

            // Obtiene el único resultado, "uniqueResult", posible (único porque "ISBN" es "UNIQUE").
            Libro libro = query.uniqueResult();

            if (libro == null){
                System.out.println("\n\n\tNo se encontró ningún libro con el ISBN {'" + isbn + "'}.");
                return;
            }

            System.out.println("\n\n\tLibro encontrado: " + libro.getTitulo() + "\n\t\tAutor: " + libro.getAutor().getNombre

            // Bloque de código para crear un nuevo ejemplar.
            Ejemplar nuevoEjemplar = new Ejemplar(codigoEjemplar, estado: EstadoEjemplar.DISPONIBLE, ubicacion: "Almacén");

            System.out.println("\n\n\tNuevo ejemplar creado/añadido: " + libro.getTitulo() + " ('" + nuevoEjemplar.getCodigoE

            // Bloque de código para añadir el nuevo ejemplar creado al libro. Aquí se emplea el método "Helper" para mantene
            libro.addEjemplar(ejemplar: nuevoEjemplar);

            // Bloque de código para guardar el autor (se guardan en cascada los libros y ejemplares).
            //session.persist(libro);
                // Con el método "Merge" actualizamos el libro (en este caso la parte de sus ejemplares correspondientes) y,
            session.merge(o: libro);
```

43:67 INS Unix (LF)

Operación 'crear ejemplar/es' (Método "agregarEjemplarALibroExistente(String isbn, String codigoEjemplar)") (GestionBiblioteca.java).

4. Paso

"Operaciones Read".

GestionBiblioteca.java

```java
186     // Crea el método "listarTodosLosAutores". Su función será ir recuperando/leyendo todos los obj
187         /*
188          * Busca y hace una lista de todos los autores registrados en la base de datos.
189          */
190     public void listarTodosLosAutores(){
191         // Se abre una sesión de Hibernate para interactuar con la base de datos.
192         session = HibernateUtil.getSessionFactory().openSession();
193         //transaction = null;
194 
195         System.out.println(x: "\n\n\tBuscando y listando a todos los autores registrados en la DB...
196 
197         try{
198             // Bloque de código para buscar todos los autores registrados en la DB mediante consult
199                     // Forma 1.
200             // List<Autor> autores = session.createQuery("FROM Autor", Autor.class).list();
201                     // Forma 2.
202             String hql = "FROM Autor";
203             Query<Autor> query = session.createQuery(string: hql, type:Autor.class);
204             List<Autor> autores = query.list();
205 
206             // Si no hay autores registrados se muestra un mensaje informativo.
207             if (autores.isEmpty()){
208                 System.out.println(x: "\n\n\tNo se encontró ningún autor. Por el momento no hay auto
209                 return;
210             }
211 
212             int contador = 1;
213 
214             System.out.println(x: "\n\n\t<==================== LISTA de AUTORES ====================
215             for (Autor autor : autores){
216                 System.out.println("\n\n\tAutor número " + contador);
217                 //System.out.println(autor);
218                 System.out.println("\n\t\tAutor: " + autor.getNombre() + " " + autor.getApellidos()
219                 System.out.println(x: "------------------------------------------------------------
220 
221                 contador ++;
222             }
223         } catch (HibernateException hex){
224             JOptionPane.showMessageDialog(parentComponent: null, "Error inesperado durante la lectura d
```

1:1 INS Unix (LF)

Operación 'listar autor/es' (Método "listarTodosLosAutores()") (GestionBiblioteca.java).

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

GestionBiblioteca.java

Source History

```java
    // Crea el método "buscarLibroPorId". Su función será ir recuperando/leyendo el objeto, libro,
        /*
         * Busca un libro por su número de identificación o ID registrado en la base de datos.
         */
    public void buscarLibroPorId(Long id){
        // Se abre una sesión de Hibernate para interactuar con la base de datos.
        session = HibernateUtil.getSessionFactory().openSession();
        //transaction = null;

        System.out.println("\n\n\tBuscando un libro por su número de identificació (ID) {" + id + "

        try{
            // Bloque de código para buscar un libro por su número de identificación (ID).
            Libro libro = session.get(type:Libro.class, srlzbl: id);

            // Si no hay libros, es decir, si la tabla "libro" está vacía se muestra un mensaje inf
            if (libro == null){
                System.out.println("\n\n\tNo se encontró o no existe ningún libro con el número de
                return;
            }

            System.out.println(x: "\n\n\t<==================== LIBRO ENCONTRADO ====================

            System.out.println("\n\tTítulo: " + libro.getTitulo() + " " + "\tNúmero de identificaci

            System.out.println(x: "\n\n\t<==================== AUTOR DEL LIBRO ====================>

            System.out.println("\n\tAutor/Escritor: " + libro.getAutor().getNombre() + " " + libro.

            System.out.println(x: "\n\n\t<==================== EJEMPLARES DEL AUTOR (Lista) ========

            List<Ejemplar> ejemplares = libro.getEjemplares();
            System.out.println("\n\tN° Libros/Novelas escritas: " + ejemplares.size() + " en total.

            if (ejemplares.isEmpty()){
                System.out.println(x: "\n\n\tNo se encontró ningún ejemplar. Por el momento no hay e
                return;
            }

```

1:1 INS Unix (LF)

Operación 'buscar libro' (Método "buscarLibroPorId(Integer idLibro)") (GestionBiblioteca.java).

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

GestionBiblioteca.java

Source History

```java
287         // Crea el método "buscarEjemplaresPorEstado". Su función será ir recuperando/leyendo todos los
288             /*
289              * Busca todos los ejemplares filtrados por su estado.
290              */
291         public void buscarEjemplaresPorEstado(EstadoEjemplar estado){
292             // Se abre una sesión de Hibernate para interactuar con la base de datos.
293             session = HibernateUtil.getSessionFactory().openSession();
294             //transaction = null;
295 
296             System.out.println("\n\n\tBuscando un ejemplar/es por su estado {" + estado + "}...");
297 
298             try{
299                 // Bloque de código para buscar un ejemplar/es por su estado usando "HQL" (Hibernate Qu
300                     // ":estado" Es el valor real que espera del parametro que pasaremos "setParameter"
301                 String hql = "FROM Ejemplar e WHERE e.estado = :estado";
302                 Query<Ejemplar> query = session.createQuery(string: hql, type:Ejemplar.class);
303                 query.setParameter(string: "estado", o: estado);
304                 List<Ejemplar> ejemplares = query.list();
305 
306                 if (ejemplares.isEmpty()){
307                     System.out.println(x: "\n\n\tNo se encontró ningún ejemplar. Por el momento no hay e
308                     return;
309                 }
310 
311                 System.out.println("\n\n\tTotal de ejemplares encontrados: " + ejemplares.size());
312 
313                 int contador = 1;
314 
315                 //for (Ejemplar ejemplar : libro.getEjemplares()){
316                 for (Ejemplar ejemplar : ejemplares){
317                     System.out.println("\n\n\tEjemplar número " + contador);
318                     //System.out.println(autor);
319                     System.out.println("\n\t\tCódigo: " + ejemplar.getCodigoEjemplar() + " " + "\tNúmer
320                     System.out.println(x: "-------------------------------------------------------------
321 
322                     contador ++;
323                 }
324             } catch (HibernateException hex){
325                 JOptionPane.showMessageDialog(parentComponent: null, "Error inesperado durante la búsqueda
```

1:1 INS Unix (LF)

Operación 'buscar ejemplar/es' (Método "buscarEjemplaresPorEstado(EstadoEjemplar estado)") (GestionBiblioteca.java).

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

GestionBiblioteca.java

Source History

```java
    // Crea el método "obtenerEstadisticasBiblioteca." Su función será ir recuperando/leyendo todos
        /*
         * Busca y trata todos los objetos de nuestra base de datos para crear y mostrar estadístic
         */
    public void obtenerEstadisticasBiblioteca(){
        // Se abre una sesión de Hibernate para interactuar con la base de datos.
        session = HibernateUtil.getSessionFactory().openSession();
        //transaction = null;

        System.out.println(x: "\n\n\tAnalizando datos y generando estadísticas de la biblioteca...")

        try{
            String hql;
            System.out.println(x: "\n\n\t<==================== ESTADÍSTICAS DE LA BIBLIOTECA =======

            // Bloque de código para contabilizar el número total de registros de autores en la BD.
                /*
                 * 1. Crea una consulta HQL que selecciona la cantidad total de autores registrados
                 * 2. Con "COUNT(a)" cuenta cuántos objetos (autores) existen en la base de datos.
                 * 3. Con "Long.class" indica que el resultado devuelto por la consulta será de tip
                 * 4. Con "uniqueResult()" obtiene un único valor (no una lista) con el total de au
                 * 5. Guarda ese número total en la variable "totalAutores".
                 */
            String autores = "SELECT COUNT(a) FROM Autor a";
            hql = autores;
            Long totalAutores = session.createQuery(string: hql, type:Long.class).uniqueResult();

            // Bloque de código para contabilizar el número total de registros de libros en la BD.
                /*
                 * 1. Crea una consulta HQL que selecciona la cantidad total de libros registrados
                 * 2. Con "COUNT(a)" cuenta cuántos objetos (libros) existen en la base de datos.
                 * 3. Con "Long.class" indica que el resultado devuelto por la consulta será de tip
                 * 4. Con "uniqueResult()" obtiene un único valor (no una lista) con el total de li
                 * 5. Guarda ese número total en la variable "totalLibros".
                 */
            String libros = "SELECT COUNT(l) FROM Libro l";
            hql = libros;
            Long totalLibros = session.createQuery(string: hql, type:Long.class).uniqueResult();
```

1:1 INS Unix (LF)

Operación 'obtener estadísticas' (Método "obtenerEstadisticasBiblioteca()") (GestionBiblioteca.java).

5. Paso

"Operaciones Update".

GestionBiblioteca.java

Source History

```java
402     // Crea el método "actualizarEstadoEjemplar." Su función será ir recuperando el/los ejemplar/es e
403         /*
404          * Busca los ejemplares por su número de identificación (ID) y actualiza su estado.
405          */
406     public void actualizarEstadoEjemplar(Integer id, EstadoEjemplar nuevoEstado){
407         // Se abre una sesión de Hibernate para interactuar con la base de datos.
408         session = HibernateUtil.getSessionFactory().openSession();
409         //transaction = null;
410
411         System.out.println("\n\n\tBuscando un ejemplar por su número de identificación o ID {" + id +
412
413         try{
414             // Se abre/inicia una nueva transacción.
415             transaction = session.beginTransaction();
416
417             // Bloque de código para buscar al ejemplar por su ID.
418             Ejemplar ejemplar = session.get(type: Ejemplar.class, srlsbl: id);
419
420             // Si no hay ejemplares, es decir, si la tabla "ejemplar" está vacía se muestra un mensaje
421             if (ejemplar == null){
422                 System.out.println("\n\n\tNo se encontró o no existe ningún ejemplar con el número de
423                 return;
424             }
425
426             System.out.println("\n\n\tEjemplar {" + id + "} encontrado exitosamente.\n\t\tLibro: " + e
427
428             // Bloque de código para actualizar el estado.
429             ejemplar.setEstado(estado: nuevoEstado);
430
431             // Si el estado del ejemplar encontrado/buscado pasa a "PRESTADO", cambia/actualiza su ubi
432             if (nuevoEstado == EstadoEjemplar.PRESTADO){
433                 ejemplar.setUbicacion(ubicacion: "En préstamo");
434             }
435
436             // Bloque de código para guardar los cambios/actualizaciones recientes hechos sobre los ca
437             session.update(o: ejemplar);
438
439             // Bloque de código para confirmar la transacción.
440             transaction.commit();
441
442             System.out.println(x: "\n\n\tEstado del ejemplar actualizado correctamente.");
443
444             System.out.println("\n\n\tEjemplar {" + id + "} encontrado exitosamente.\n\t\tLibro: " + e
```

512:119 INS Unix (LF)

Operación 'actualizar estado ejemplar/es autor/es' (Método "actualizarEstadoEjemplar(Integer idEjemplar, EstadoEjemplar nuevoEstado)") (GestionBiblioteca.java).

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

GestionBiblioteca.java

Source History

```java
453
454         // Crea el método "actualizarDatosLibro." Su función será ir recuperando el/los libro/s e ir actua
455             /*
456              * Busca los libros por su número de identificación (ID) y actualiza sus datos.
457              */
458         public void actualizarDatosLibro(Integer id, String nuevoTitulo, Integer nuevasPaginas){
459             // Se abre una sesión de Hibernate para interactuar con la base de datos.
460             session = HibernateUtil.getSessionFactory().openSession();
461             //transaction = null;
462
463             System.out.println("\n\n\tBuscando un libro por su número de identificación o ID {" + id + "}.
464
465             try{
466                 // Se abre/inicia una nueva transacción.
467                 transaction = session.beginTransaction();
468
469                 // Bloque de código para buscar al libro por su ID.
470                 Libro libro = session.get(type: Libro.class, srlsbl: id);
471
472                 // Si no hay libros, es decir, si la tabla "libro" está vacía se muestra un mensaje inform
473                 if (libro == null){
474                     System.out.println("\n\n\tNo se encontró o no existe ningún libro con el número de ide
475                     return;
476                 }
477
478                 System.out.println("\n\n\tLibro {" + id + "} encontrado exitosamente.\n\t\tCódigo ISBN: "
479
480                 // Bloque de código para actualizar los campos/atributos del libro.
481                 libro.setTitulo(titulo: nuevoTitulo);
482                 libro.setNumeroPaginas(numeroPaginas: nuevasPaginas);
483
484                 // Bloque de código para guardar los cambios/actualizaciones recientes hechos sobre los ca
485                 session.update(o: libro);
486
487                 // Bloque de código para confirmar la transacción.
488                 transaction.commit();
489
490                 System.out.println(x: "\n\n\tCampos/Atributos (título y número de páginas) actualizado corr
491
492                 System.out.println("\n\n\tLibro {" + id + "} encontrado exitosamente.\n\t\tCódigo ISBN: "
493             } catch (HibernateException hex){
494                 // Si ocurre un error/excepción revierte el proceso de transacción.
495                 if (transaction != null) transaction.rollback();
496                 JOptionPane.showMessageDialog(parentComponent:null, "Error inesperado durante la actualización
```

512:119 INS Unix (LF)

Operación 'actualizar estado ejemplar/es autor/es' (Método "actualizarDatosLibro(Integer idLibro, String nuevoTitulo, Integer nuevasPaginas)") (GestionBiblioteca.java).

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

GestionBiblioteca.java

Source History

```java
    // Crea el método "transferirLibrosEntreAutores." Su función será transferir todos los datos de un
        /*
         * Busca los autores (origen y destino) por su número de identificación (ID) y transfiere sus
         */
    public void transferirLibrosEntreAutores(Integer idAutorOrigen, Integer idAutorDestino){
        // Se abre una sesión de Hibernate para interactuar con la base de datos.
        session = HibernateUtil.getSessionFactory().openSession();
        //transaction = null;

        System.out.println("\n\n\tBuscando autor (de origen) por su número de identificación o ID {" +
        System.out.println("\n\n\tBuscando autor (de destino) por su número de identificación o ID {"

        try{
            // Se abre/inicia una nueva transacción.
            transaction = session.beginTransaction();

            // Bloque de código para buscar a ambos autores por su ID.
            Autor autorOrigen = session.get(type: Autor.class, srlsbl: idAutorOrigen);
            Autor autorDestino = session.get(type: Autor.class, srlsbl: idAutorDestino);

            // Si no hay ninguno de los autores, es decir, si la tabla "autor" está vacía se muestra u
            if (autorOrigen == null){
                System.out.println("\n\n\tNo se encontró o no existe ningún autor (de origen) con el n
                return;
            }
            if (autorDestino == null){
                System.out.println("\n\n\tNo se encontró o no existe ningún autor (de destio) con el n
                return;
            }

            System.out.println("\n\n\tAutor (de origen) {" + idAutorOrigen + "} encontrado exitosament
            System.out.println("\n\n\tAutor (de destino) {" + idAutorDestino + "} encontrado exitosame

            // Bloque de código para obtener una lista con los libros del autor de origen.
            List<Libro> librosAtraspasar = autorOrigen.getLibros();

            // Si no hay libros, es decir, si la tabla "libro" está vacía se muestra un mensaje inform
            if (librosAtraspasar.isEmpty()){
            System.out.println(x: "\n\n\tEl autor (de origen) no tiene libros para transferir");
                return;
            }

            System.out.println("\n\n\tEl autor (de origen) si tiene libros para transferir.\n\t\tLibro
```

512:119 INS Unix (LF)

Operación 'actualizar estado ejemplar/es autor/es' (Método "transferirLibrosEntreAutores(Integer idAutorOrigen, Integer idAutorDestino)") (GestionBiblioteca.java).

6. Paso

"Operaciones Delete".

GestionBiblioteca.java

Source History

```java
564
565     // Crea el método "eliminarEjemplar." Su función será recuperar el libro para eliminar su ejemplar.
566         /*
567          * Busca el libro a partir del código de identificación (ID) del ejemplar para después borrar ese r
568          */
569     public void eliminarEjemplar(Integer id){
570         // Se abre una sesión de Hibernate para interactuar con la base de datos.
571         session = HibernateUtil.getSessionFactory().openSession();
572         //transaction = null;
573
574         System.out.println("\n\n\tBuscando ejemplar por su número de identificación o ID {" + id + "}...").
575
576         try{
577             // Se abre/inicia una nueva transacción.
578             transaction = session.beginTransaction();
579
580             // Bloque de código para buscar ejemplares por su ID.
581             Ejemplar ejemplar = session.get(Ejemplar.class, id);
582
583             // Si no hay ejemplares, es decir, si la tabla "ejemplar" está vacía se muestra un mensaje info
584             if (ejemplar == null){
585                 System.out.println("\n\n\tNo se encontró o no existe ningún ejemplar con el número de ident
586                 return;
587             }
588
589             System.out.println("\n\n\tEjemplar {" + id + "} encontrado exitosamente.\n\t\tCódigo: " + ejemp
590
591             // Bloque de código para obtener el libro a partir del ejemplar.
592             Libro libro = ejemplar.getLibro();
593
594             // Bloque de código para mantener consistencia bidireccional entre "Libro" y "Ejemplar".
595             libro.removeEjemplar(ejemplar); /* Llamamos al método "helper" → "removeEjemplar" para consegu
596
597             // Bloque de código para confirmar la transacción.
598             transaction.commit();
599
600             System.out.println("\n\n\tEjemplar eliminado/borrado de la lista de ejemplares del libro ('" +
601         } catch (HibernateException hex){
602             // Si ocurre un error/excepción revierte el proceso de transacción.
603             if (transaction != null) transaction.rollback();
604             JOptionPane.showMessageDialog(null, "Error inesperado durante la ejecución de la transacción (e
605         } finally{
606             session.close();
607         }
608     }
609
610     // Crea el método "eliminarLibro." Su función será recuperar el autor para eliminar su libro.
611         /*
```

708:2 INS Unix (LF)

Operación 'actualizar estado ejemplar/es autor/es' (Método "eliminarEjemplar(Integer idEjemplar)") (GestionBiblioteca.java).

GestionBiblioteca.java

Source History

```java
    // Crea el método "eliminarLibro." Su función será recuperar el autor para eliminar su libro.
        /*
         * Busca el autor a partir del código de identificación (ID) del libro para después borrar ese mis
         */
    public void eliminarLibro(Integer id){
        // Se abre una sesión de Hibernate para interactuar con la base de datos.
        session = HibernateUtil.getSessionFactory().openSession();
        //transaction = null;

        System.out.println("\n\n\tBuscando libro por su número de identificación o ID {" + id + "}...");

        try{
            // Se abre/inicia una nueva transacción.
            transaction = session.beginTransaction();

            // Bloque de código para buscar el libro por su ID.
            Libro libro = session.get(Libro.class, id);

            // Si no hay libros, es decir, si la tabla "libro" está vacía se muestra un mensaje informativ
            if (libro == null){
                System.out.println("\n\n\tNo se encontró o no existe ningún libro con el número de identif
                return;
            }

            System.out.println("\n\n\tLibro {" + id + "} encontrado exitosamente.\n\t\tCódigo ISBN: " + li

            // Bloque de código (opcional) para contabilizar y así saber cúantos ejemplares serán eliminad
            int ejemplaresEliminados = libro.getEjemplares().size();

            // Bloque de código para obtener el autor.
            Autor autor = libro.getAutor();

            // Bloque de código para mantener consistencia bidireccional entre "Autor" y "Libro".
            autor.removeLibro(libro); /* Llamamos al método "helper" → "removeLibro" para conseguir elimin

            // Bloque de código para confirmar la transacción.
            transaction.commit();

            System.out.println("\n\n\tLibro eliminado/borrado de la lista de libros del autor ('" + libro.
        } catch (HibernateException hex){
            // Si ocurre un error/excepción revierte el proceso de transacción.
            if (transaction != null) transaction.rollback();
            JOptionPane.showMessageDialog(null, "Error inesperado durante la ejecución de la transacción (
        } finally{
            session.close();
        }
    }
```

708:2 INS

Operación 'actualizar estado ejemplar/es autor/es' (Método "eliminarLibro(Integer idLibro)") (GestionBiblioteca.java).

Actividad 3.10. (AED) - Actividad: Biblioteca

Profesor: Javier Sebastián Herrero **Módulo:** Acceso a datos (AED)

GestionBiblioteca.java

Source History

```java
658     // Crea el método "eliminarAutorConCascada." Su función será recuperar y eliminar el autor en cascada.
659         /*
660          * Busca el autor a partir del código de identificación (ID) para después eliminarlo/borrarlo en ca
661          */
662     public void eliminarAutorConCascada(Integer id){
663         // Se abre una sesión de Hibernate para interactuar con la base de datos.
664         session = HibernateUtil.getSessionFactory().openSession();
665         //transaction = null;
666
667         System.out.println("\n\n\tBuscando libro por su número de identificación o ID {" + id + "}...");
668
669         try{
670             // Se abre/inicia una nueva transacción.
671             transaction = session.beginTransaction();
672
673             // Bloque de código para buscar el autor.
674             Autor autor = session.get(Autor.class, id);
675
676             // Si no hay autores, es decir, si la tabla "autor" está vacía se muestra un mensaje informati
677             if (autor == null){
678                 System.out.println("\n\n\tNo se encontró o no existe ningún autor con el número de identif
679                 return;
680             }
681
682             System.out.println("\n\n\tAutor {" + id + "} encontrado exitosamente.\n\t\tNombre: " + autor.ge
683
684             // Bloque de código (opcional) para contabilizar y así saber cúantos libros serán eliminados a
685             int librosEliminados = autor.getLibros().size();
686
687             // Bloque de código (opcional) para contabilizar y así saber cúantos ejemplares serán eliminad
688             int ejemplaresEliminados = 0;
689             for (Libro l : autor.getLibros()){
690                 ejemplaresEliminados += l.getEjemplares().size();
691             }
692
693             // Bloque de código para mantener el borrado en cascada entre "Ejemplar", "Libro" y "Autor".
694             session.remove(autor);
695
696             // Bloque de código para confirmar la transacción.
697             transaction.commit();
698
699             System.out.println("\n\n\tAutor eliminado/borrado de la lista de autores ('" + autor.getNombre
700         } catch (HibernateException hex){
701             // Si ocurre un error/excepción revierte el proceso de transacción.
702             if (transaction != null) transaction.rollback();
703             JOptionPane.showMessageDialog(null, "Error inesperado durante la ejecución de la transacción (
704         } finally{
705             session.close();
```

708:2 INS Unix (LF)

Operación 'actualizar estado ejemplar/es autor/es' (Método "eliminarAutorConCascada(Integer idAutor)") (GestionBiblioteca.java).

7. Paso

"Operaciones Update".

GestionBiblioteca.java × Main.java ×

Source History

```java
// Crea la clase 'main', principal del programa.
public class Main{
    // Crea el escáner para leer datos introducidos por teclado, por el usuario.
    private static final Scanner user = new Scanner(System.in);

    // Crea el objeto "gestor" que contiene todas las operaciones CRUD importadas del archivo "GestionBib
    private static final GestionBiblioteca gestor = new GestionBiblioteca();

    // Crea el método 'main', principal del programa.
    public static void main(String[] args){
        boolean continuar = true;

        // Bucle "do-while" que hará de bucle principal del menú.
        do{
            try{
                // Llamamos/Inicializamos el método "mostrarMenuPrincipal".
                mostrarMenuPrincipal();

                int opcion = leerOpcion();

                // Condicional "switch" para las opciones.
                switch (opcion){
                    // ========== OPERACIONES CREATE ==========
                    /*
                    case 1:
                        // Inserta autores, libros y ejemplares de prueba (apartado 3).
                        gestor.insertarDatosIniciales();
                        break;
                    */
                    case 1 -> crearAutorConLibros();
                    case 2 -> agregarEjemplarALibroExistente();

                    // ========== OPERACIONES READ ==========
                    case 3 -> listarTodosLosAutores();
                    case 4 -> buscarLibroPorId();
                    case 5 -> buscarEjemplaresPorEstado();
                    case 6 -> obtenerEstadisticasBiblioteca();

                    // ========== OPERACIONES UPDATE ==========
                    case 7 -> actualizarEstadoEjemplar();
                    case 8 -> actualizarDatosLibro();
                    case 9 -> transferirLibrosEntreAutores();

                    // ========== OPERACIONES DELETE ==========
                    case 10 -> eliminarEjemplar();
                    case 11 -> eliminarLibro();
                    case 12 -> eliminarAutorConCascada();
```

3:19 INS Windows (CRLF)

8. Paso

"Explicación del programa".

Para empezar a desarrollar esta última parte del proyecto primero hay que dar pie a la estructura del mismo:

```
BibliotecaHibernate/
├── src/
│   └── main/
│       ├── java/
│       │   └── com/
│       │       └── dam/
│       │           ├── modelo/
│       │           │   ├── Autor.java
│       │           │   ├── Libro.java
│       │           │   └── Ejemplar.java
│       │           ├── util/
│       │           │   └── HibernateUtil.java
│       │           ├── gestion/
│       │           │   └── GestionBiblioteca.java
│       │           └── Main.java
│       └── resources/
│           └── hibernate.cfg.xml
└── pom.xml
```

Una vez hecho esto, podremos entender cómo es el funcionamiento de nuestra aplicación. Empezando por el "pom.xml" que es el archivo de configuración de Maven donde definimos todas las dependencias necesarias para el proyecto. En este archivo hemos incluido tres dependencias:

- "MySQL Connector - J" para la conexión con la base de datos.
- "Hibernate Core" para el mapeo objeto-relacional.
- "Javax Persistence API (JPA)" que proporciona las anotaciones JPA que utilizaremos en nuestras entidades.

Por otro lado, Maven se encargará automáticamente de descargar estas librerías y gestionar sus dependencias transitivas, permitiéndonos centrarnos en el desarrollo sin preocuparnos por la gestión manual de las librerías.

Siguiendo con el desarrollo de la app tenemos “hibernate.cfg.xml” contiene toda la configuración de Hibernate. Aquí especificamos los parámetros de conexión a la base de datos MySQL (URL, usuario, contraseña, etc.). También definimos el dialecto de MySQL (dialecto SQL) que Hibernate utilizará para generar las consultas SQL optimizadas para este motor de base de datos. Además, hemos activado propiedades de depuración como “show_sql” y “format_sql” que nos permiten visualizar en la consola todas las sentencias SQL que Hibernate genera automáticamente, facilitando el seguimiento y la comprensión de las operaciones realizadas. Posteriormente definiremos “hbm2ddl.auto” configurándola en “validate”, con ello conseguiremos validar/permitir la gestión automática del esquema de la base de datos cuando se detecten cambios en las entidades. Finalmente, en este archivo mapeamos las tres entidades principales del proyecto mediante etiquetas “mapping class", indicando a Hibernate qué clases debe gestionar.

`Por otro lado, la clase “HibernateUtil.java” implementa el patrón de diseño “Singleton” para gestionar el “SessionFactory” de Hibernate. Esta clase contiene un bloque estático, “static”, que se ejecuta al cargar la clase, creando el “SessionFactory” a partir de la configuración definida en “hibernate.cfg.xml”. El método “getSessionFactory()” proporciona acceso a dicha instancia única desde cualquier parte de la aplicación, mientras que el método “shutdown()” se encarga de cerrar correctamente el objeto, el “SessionFactory”, cuando la aplicación finaliza, liberando todos los recursos y conexiones con la base de datos. En definitiva, esto permite garantizar un uso eficiente de los recursos y evitar problemas de concurrencia.

Siguiendo con la estructura del proyecto, encontramos el paquete “modelo”. Dentro de dicho paquete se hallan tres clases java, cada una representando a una entidad del programa/sistema:

- “Autor.java” es una de las tres entidades y representa a los escritores. Está mapeada a la tabla “autores” mediante la anotación “@Entity”. Cada atributo de la clase se mapea a una columna de la tabla utilizando “@Column”, mientras que el campo “id” está marcado con “@Id” como clave primaria y “@GeneratedValue” para indicar que su valor se genera automáticamente. La característica más importante de esta clase es su relación “@OneToMany” con la entidad “Libro”, configurada con “cascade = CascadeType.ALL” y “orphanRemoval = true”, lo que significa e implica que todas y cada una de las operaciones realizadas sobre un autor se propagan automáticamente a sus libros, y que los libros sin autor asociado se eliminan automáticamente. De igual forma, los métodos “helper” “addLibro()” y “removeLibro()” son fundamentales para mantener la consistencia bidireccional de esta relación, actualizando tanto la lista de libros en el autor como la referencia del autor en cada libro.

- "Libro.java" es la segunda de las tres entidades y representa a los libros escritos. Es una entidad clave para el proyecto, pues es la posición central del modelo, estableciendo relaciones para ambas direcciones, tanto para "Autor" con "@ManyToOne" (para cargar el autor sólo cuando sea necesario, optimizando así el rendimiento) como para "Ejemplar" con "@OneToMany" (también configurada con cascada completa para conseguir esa bidireccionalidad y concurrencia de datos). Por otro lado, su mapeo se rige a la tabla "libros" y tiene una estructura de anotaciones similar a la de "Autor", con la diferencia de campos presentes como "isbn" que está marcada como "unique = true", garantizando que no puedan existir dos libros con el mismo código ISBN en la base de datos. Esta clase también implementa métodos "helper" "addEjemplar()" y "removeEjemplar()" que mantienen la coherencia entre la lista de ejemplares del libro y la referencia del libro con/para cada ejemplar al que corresponde.

- "Ejemplar.java" es la tercera y última de las tres entidades y representa a los ejemplares, las copias físicas y oficiales de los libros originales escritos por el autor. A diferencia de las anteriores entidades, ésta contiene un método "enum", una enumeración interna con la que representar y designar a cada ejemplar uno de los estados que define los estados posibles dentro de la biblioteca, "EstadoEjemplar" (DISPONIBLE, PRESTADO, REPARACION y BAJA). Éste método al igual que en el resto de entidades está mapeado mediante "@Enumerated(EnumType.STRING)", lo que hace que Hibernate almacene el nombre del "enum" como cadena de texto en lugar de su posición ordinal, proporcionando mayor legibilidad y evitando problemas en caso de que se reordene, altere su estado/valor, el "enum" en el futuro. Es una entidad con relación "@ManyToOne" con "Libro" utiliza también carga perezosa "LAZY", lo que significa que el libro asociado solo se cargará de la base de datos cuando se acceda explícitamente al método "getLibro()".

La clase “GestionBiblioteca.java” contiene toda la lógica funcional del programa, todos los métodos/operaciones CRUD del sistema. Cada método maneja su propia transacción siguiendo el patrón “try-with-resources” para garantizar que las sesiones de Hibernate se cierren automáticamente, incluso si ocurre una excepción/error durante la ejecución. Los métodos de creación “Operaciones Create” demuestran el poder de las operaciones en cascada: al persistir un autor con “session.persist(autor)”, Hibernate se encarga automáticamente de guardar todos sus libros y los ejemplares de cada libro, reduciendo significativamente la cantidad de código necesario, ergo libera carga significativa al código. Los métodos de consulta “Operaciones Read” utilizan sentencias HQL (Hibernate Query Language), un lenguaje orientado a objetos similar a SQL pero que trabaja con clases y atributos en lugar de tablas y columnas, es decir, trabaja directamente con objetos y no con entidades de tablas y columnas. Por ejemplo, la consulta “FROM Ejemplar e WHERE e.estado = :estado” recupera ejemplares filtrando por su estado, e Hibernate la traduce automáticamente al SQL apropiado para MySQL. Los métodos de actualización “Operaciones Update” emplean “session.merge()” para entidades existentes, e Hibernate aplica su mecanismo de "dirty checking" para detectar/saber qué campos han cambiado y así poder generar sentencias “UPDATE” que solo modifican dichos campos específicos. Los métodos de eliminación “Operaciones Delete” demuestran el funcionamiento completo de las cascadas. Es decir, eliminar un autor provoca la eliminación automática de todos sus libros, y estos a su vez eliminan todos sus ejemplares, todo ello en una sola operación que mantiene la integridad referencial de la base de datos.

La clase “Main.java” proporciona la pieza restante del código, una interfaz de usuario mediante un menú interactivo por consola para una interacción constante con el usuario. Este menú está organizado en cuatro secciones claramente diferenciadas: ‘operaciones CREATE’ (opciones 1-2), ‘operaciones READ’ (3-6), ‘operaciones UPDATE’ (7-9) y ‘operaciones DELETE’ (10-12), además de una última opción con la que poder salir del programa (0). La aplicación se ejecuta en un bucle “do-while” que solo termina cuando el usuario selecciona la opción de salida. La gestión de errores es robusta, capturando “InputMismatchException” cuando el usuario introduce valores no esperados o incorrectos, como lo son los no numéricos y limpiando el buffer del “Scanner user” para evitar bucles infinitos. Las operaciones destructivas “DELETE” solicitan confirmación explícita del usuario antes de proceder, argumentando o explicando lo que conllevaría llevar a cabo la acción de ejecutar dicha operación, mostrando advertencias sobre las consecuencias de la cascada. Al finalizar, el método “salir()” invoca “HibernateUtil.shutdown()” para cerrar correctamente el “SessionFactory” y liberar todos los recursos, garantizando una terminación limpia de la aplicación. Este diseño ofrece una experiencia de usuario clara y segura, permitiendo probar todas las funcionalidades del sistema de forma intuitiva.

Por otro lado, a lo largo del código (de todos los archivos) encontrará comentarios claros, limpios, intuitivos de entender e indicativos sobre qué está pasando en ese momento, qué ocurre, para qué es la línea de código, que función cumple el método o la clase, etc.

En cuanto a las dificultades encontradas durante el desarrollo y trabajo de esta práctica me gustaría resaltar un par de cosas. Empezando por entender cómo sería el funcionamiento del programa dada la indicación explícita en la tarea de que hubiese bidireccionalidad entre tablas. Este conflicto se resolvió en clase tras la explicación del profesor, de Javier.

Otra cuestión fue la implementación de nuevos métodos como los "helper" con los que mantener un programa bidireccional y con concurrencia de datos. Para ello lo que hice fue ir trasteando en un proyecto independiente/ajeno a este con la ayuda de dos AI's (ChatGPT y Claude) para tener dos puntos de referencia y no basarme en la primera respuesta que me diese solo una de ellas.

Si bien es cierto que usé la ambas AI a lo largo de todo el proyecto, mi forma de trabajar era la siguiente, primero leía e intentaba hacerlo yo, después mandaba el "prompt" de la parte que estaba trabajando (no de todo el archivo o proyecto) y comparaba las dos respuestas dadas y lo combinaba con lo que ya tenía hecho/avanzado. También me he apoyado bastante en esta/s herramienta/s para los comentarios, pidiéndoles que me explicasen una parte concreta del código, una vez leída y entendida la explicación, trasladaba de forma más resumida, propia y escueta lo que yo había entendido de la explicación.

Y finalmente, también me gustaría hacer hincapié en el uso de HIbernate teniendo en cuenta que es un procedimiento relativamente nuevo para nosotros. Desde mi punto de vista, el uso de este sistema no solo me ha permitido trabajar más fácilmente, sino que me ha sido una metodología mucho más fácil y rápida de comprender y trabajar a diferencia del método que aplicábamos antes que era únicamente realizar la correspondencia objeto-relacional mediante archivos "XML".